🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

รายวิชา: CP352005 Networks

กลุ่ม: 19

รายชื่อสมาชิก:

673380265-8 ชาคริต ภักมี (Part 1& Part2)

673380049-4 ปิยชญานินท์ โทนะพันธ์

673380291-7 วิภู หากันได้

673380063-0 ศุภเชฏฐ์ วอหล้า

ลำดับ	ชื่อ-นามสกุล	รหัสนักศึกษา	บทบาท (Role)
1	ชาคริต ภักมี	673380265-8	Network Architect/IaC/DevOps
2			
3			
4			
5			

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

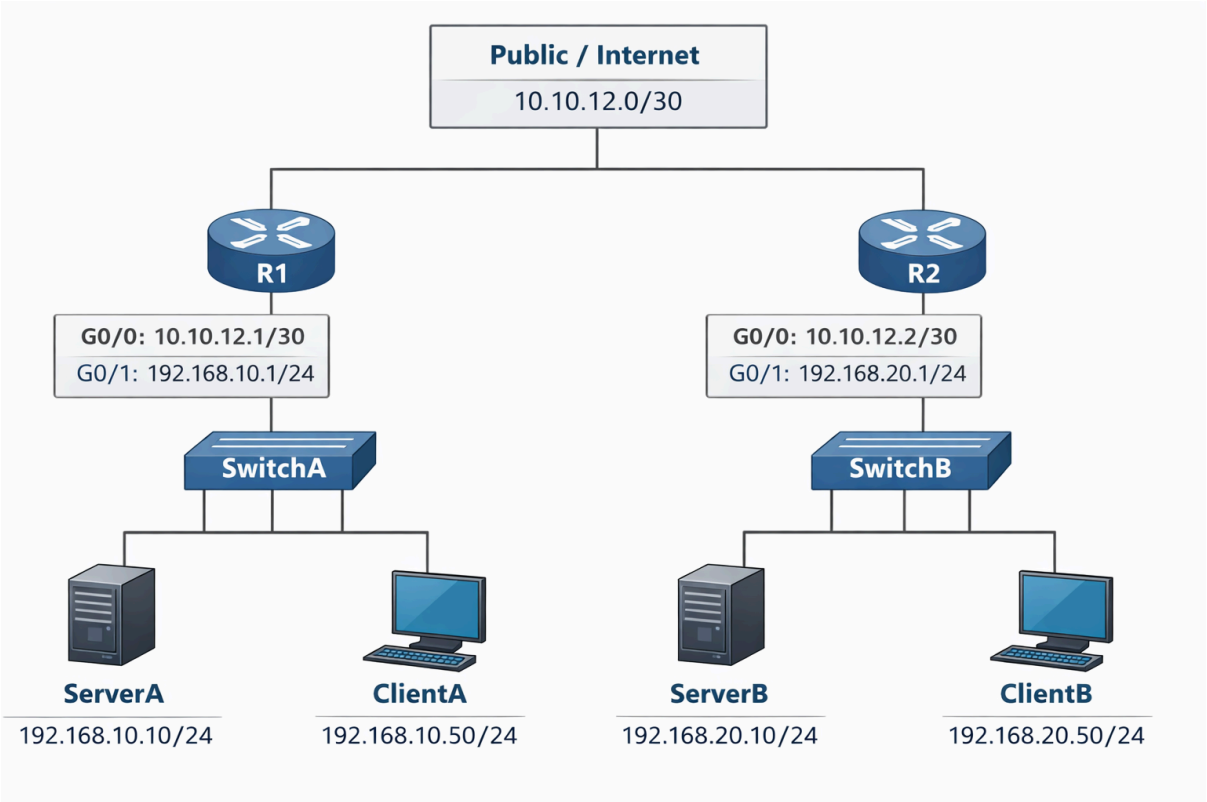
👥 TEAM BREAKDOWN (5–6 students)

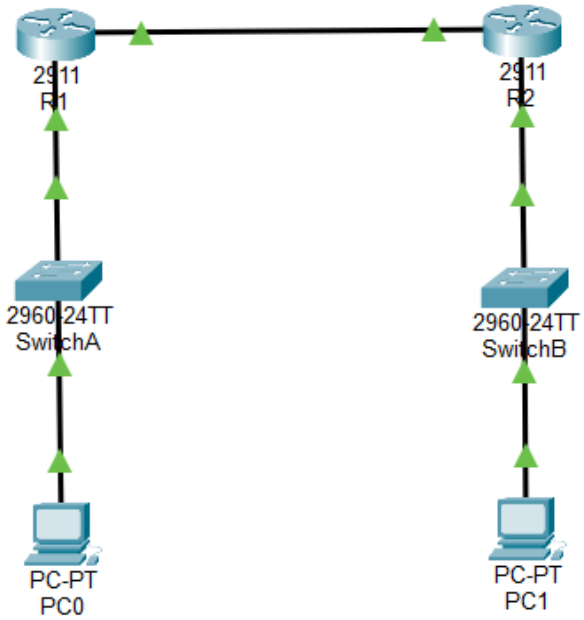
Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

```python
                10.10.12.0/30 “Public/Internet”
             ┌──────────────────────────┐
             |                          |
         R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
             |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
             |                          |
         SwitchA                     SwitchB
             |                          |
 ServerA (mockup)           ServerB (mockup)
 ClientA                        ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.





https://drive.google.com/file/d/1a6rg6bJLabbudKHrE8vx_1oRx8DRIg1r/view?usp=drive_link

---

📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public

Component	Interface	IP	Mask	Role
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1

interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
```

```
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#
R1# conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface g0/0/1
R1(config-if)#ip address 10.10.12.1 255.255.255.252
% 10.10.12.0 overlaps with GigabitEthernet0/0/0
R1(config-if)#interface g0/0/0
R1(config-if)#ip address 10.10.12.1 255.255.255.252
R1(config-if)#ip nat outside
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#interface g0/0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#ip nat inside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

R1(config-if)#exit
R1(config)#ip route 192.168.20.0 255.255.255.0 10.10.12.2
R1(config)#access-list 10 permit 192.168.10.0 0.0.0.255
R1(config)#ip nat inside source list 10 interface g0/0/0 overload
R1(config)#ip nat inside source static tcp 192.168.20.10 80 10.10.12.2 80
R1(config)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console
write memory
Building configuration...
[OK]
R1#
```

สรุปการตั้งค่าเราเตอร์ R1:

- **Interfaces**: กำหนด IP ให้ g0/0/0 (10.10.12.1) เป็นฝั่ง **NAT Outside** และ g0/0/1 (192.168.10.1) เป็นฝั่ง **NAT Inside**
- **Routing**: เพิ่ม **Static Route** ไปยังวงเครือข่าย 192.168.20.0 ผ่าน IP 10.10.12.2
- **NAT**: ตั้งค่า **NAT Overload** (PAT) ผ่าน ACL 10 เพื่อให้วง LAN ออกอินเทอร์เน็ตได้ และทำ **Static NAT** (Port Forwarding) สำหรับบริการ HTTP (พอร์ต 80)
- **Save**: บันทึกการตั้งค่าลงหน่วยความจำด้วยคำสั่ง

🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t
```

```
hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
Press RETURN to get started!



Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R2
R2(config)#interface g0/0/0
R2(config-if)#ip address 10.10.12.2 255.255.255.252
R2(config-if)#ip nat outside
R2(config-if)#no shutdown

R2(config-if)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/0, changed state to down
exit
R2(config)#interface g0/0/1
R2(config-if)#ip address 192.168.20.1 255.255.255.0
R2(config-if)#ip nat inside
R2(config-if)#no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up
exit
R2(config)#ip route 192.168.10.0 255.255.255.0 10.10.12.1
R2(config)#access-list 10 permit 192.168.20.0 0.0.0.255
R2(config)#ip nat inside source list 10 interface g0/0/0 overload
R2(config)#ip nat inside source static tcp 192.168.20.10 80 10.10.12.2 80
R2(config)#end
R2#
%SYS-5-CONFIG_I: Configured from console by console
write memory
Building configuration...
[OK]
R2#
```

สรุปการกำหนดค่าเราเตอร์ R2:

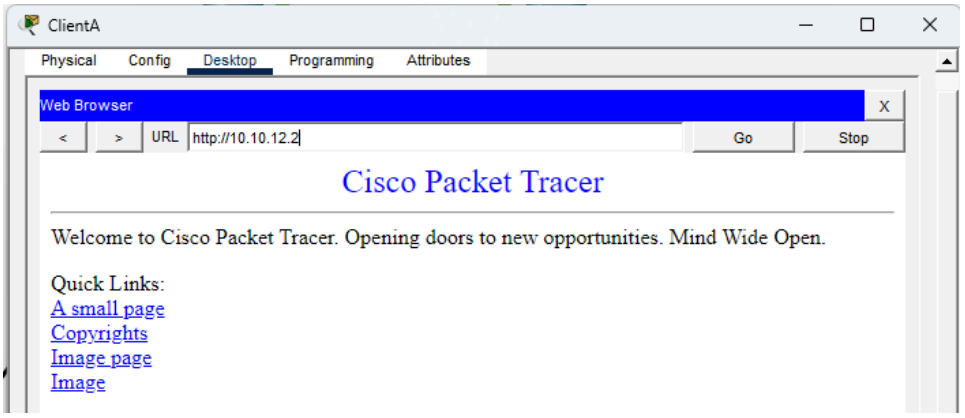
- **Interfaces**:
  - `g0/0/0`: ตั้งค่า IP `10.10.12.2` (Public) และกำหนดเป็น `ip nat outside`.
  - `g0/0/1`: ตั้งค่า IP `192.168.20.1` (LAN B) และกำหนดเป็น `ip nat inside`.

- **Routing**: เพิ่ม **Static Route** กลับไปยังวง LAN A (`192.168.10.0`) ผ่านทาง IP `10.10.12.1`.
- **NAT**:
  - **NAT Overload**: ใช้ ACL 10 เพื่อให้คนใน LAN B ออกสู่อินเทอร์เน็ตจำลองได้.
  - **Static NAT**: ทำ Port Forwarding พอร์ต 80 จาก IP ภายนอกมายัง ServerB (`192.168.20.10`).
- **Save**: บันทึกการตั้งค่าเรียบร้อยด้วยคำสั่ง `write memory`

🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
None
show ip route
show ip nat translations
```



- **HTTP Connectivity**: ClientA สามารถเข้าถึงหน้าเว็บของ **Cisco Packet Tracer** ผ่าน URL http://10.10.12.2 ได้สำเร็จ
- **NAT & Routing Verification**: ผลลัพธ์นี้พิสูจน์ว่า **Static Route** และ **Static NAT (Port Forwarding)** ที่ตั้งค่าไว้บนเราเตอร์ทำงานถูกต้อง เนื่องจาก ClientA สามารถเรียกข้อมูลข้ามวงเครือข่ายจาก LAN A ไปยัง Server ใน LAN B ได้ผ่าน Public IP

```
R2(config)#exit
R2#
%SYS-5-CONFIG_I: Configured from console by console
write memory
Building configuration...
[OK]
R2#
R2#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   10.10.12.2      YES manual down                  down
GigabitEthernet0/0/1   192.168.20.1    YES manual up                    up
GigabitEthernet0/0/2   unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R2#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to
up

R2#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   10.10.12.2      YES manual up                    up
GigabitEthernet0/0/1   192.168.20.1    YES manual up                    up
GigabitEthernet0/0/2   unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R2#show ip nat translations
Pro  Inside global     Inside local       Outside local      Outside global
tcp 10.10.12.2:80      192.168.20.10:80   ---                ---
tcp 10.10.12.2:80      192.168.20.10:80   10.10.12.1:1030    10.10.12.1:1030

R2#
```

1.ตรวจสอบสถานะพอร์ต (show ip interface brief)

- **GigabitEthernet0/0/0**: มี IP 10.10.12.2 และสถานะเป็น **up/up** (เชื่อมต่อกับเครือข่าย Public สำเร็จ)
- **GigabitEthernet0/0/1**: มี IP 192.168.20.1 และสถานะเป็น **up/up** (เชื่อมต่อกับ LAN B สำเร็จ)

2.ตรวจสอบการแปลงที่อยู่ (show ip nat translations)

ตารางนี้แสดงให้เห็นว่า **Static NAT (Port Forwarding)** และ **NAT Overload** กำลังทำงาน:

- ส่งข้อมูลจาก **Inside Local** (192.168.20.10:80 ซึ่งคือ ServerB) ออกไปยัง **Outside Global** ผ่านพอร์ตที่กำหนด
- ยืนยันว่าเครื่องจากวงอื่น (เช่น ClientA) สามารถเข้าถึง Service ของ ServerB ได้ผ่าน IP ส่วนกลาง 10.10.12.2

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 10.10.12.2

Pinging 10.10.12.2 with 32 bytes of data:

Reply from 10.10.12.2: bytes=32 time<1ms TTL=254
Reply from 10.10.12.2: bytes=32 time<1ms TTL=254
Reply from 10.10.12.2: bytes=32 time<1ms TTL=254
Reply from 10.10.12.2: bytes=32 time<1ms TTL=254

Ping statistics for 10.10.12.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

ผลการทดสอบ:

- ได้รับสัญญาณตอบรับ (**Reply**) ครบถ้วนทั้ง 4 ครั้ง
- **0% loss**: ไม่ สูญเสียข้อมูลระหว่างทาง
- **Latency**: ความเร็วในการตอบสนองต่ำกว่า 1ms แสดงถึงการเชื่อมต่อที่เสถียรมาก

ข้อสรุป: ยืนยันว่าการตั้งค่า **Routing** บน R1 และสถานะอินเทอร์เฟซของ R2 ทำงานได้ถูกต้อง

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPO

```shell
git clone https://github.com/boonsup/automation.git
cd automation
```

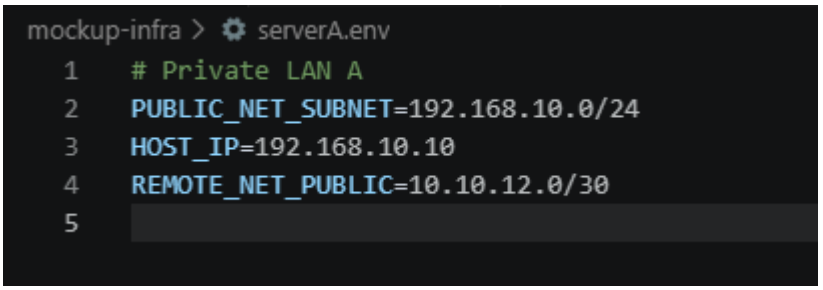
🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```



เปิดไฟล์ **ServerA.env** เพื่อกำหนดค่าเครือข่ายให้ **ServerA**:

- **PUBLIC_NET_SUBNET**: กำหนดวง LAN A เป็น 192.168.10.0/24.
- **HOST_IP**: กำหนด IP ของเซิร์ฟเวอร์ตัวนี้เป็น 192.168.10.10.
- **REMOTE_NET_PUBLIC**: ระบุวงเครือข่ายสาธารณะที่ใช้เชื่อมต่อคือ 10.10.12.0/30.

ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

```
mockup-infra > serverB.env
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

เปิดไฟล์ **ServerB.env** เพื่อกำหนดค่าเครือข่ายให้ **ServerB**:

- **PUBLIC_NET_SUBNET**: กำหนดวง LAN A เป็น 192.168.20.0/24.
- **HOST_IP**: กำหนด IP ของเซิร์ฟเวอร์ตัวนี้เป็น 192.168.20.10.
- **REMOTE_NET_PUBLIC**: ระบุวงเครือข่ายสาธารณะที่ใช้เชื่อมต่อคือ 10.10.12.0/30.

🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

None

```
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

```
🔧 Initializing Mockup Infrastructure...
==================================================

🔐 [Layer 5/6 - Session] Generating TLS Certificates...
/home/sox/Documents/Network/automation/mockup-infra/manage.py:118: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ver
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_before(datetime.utcnow())
/home/sox/Documents/Network/automation/mockup-infra/manage.py:119: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ver
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_after(datetime.utcnow() + timedelta(days=365))
✅ Certificates generated successfully:
   - Private Key: /home/sox/Documents/Network/automation/mockup-infra/certs/server.key
   - Certificate: /home/sox/Documents/Network/automation/mockup-infra/certs/server.crt
   - Common Name: api.mockup.test
   - Subject Alt Names: localhost, api.mockup.test, 127.0.0.1, 172.18.0.1, 172.19.0.1

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
[
    {
        "name": "private_net",
        "id": "8029c0a983f7e4f3f5e0b1689fc2ae98ee33e93e1f49de58df0db90bac79683a",
        "driver": "bridge",
        "network_interface": "podman2",
        "created": "2026-03-04T20:37:40.121546148+07:00",
        "subnets": [
            {
                "subnet": "10.89.1.0/24",
                "gateway": "10.89.1.1"
            }
        ],
        "ipv6_enabled": false,
        "internal": true,
        "dns_enabled": true,
        "ipam_options": {
            "driver": "host-local"
        }
    }
]
✅ Private network is isolated (internal: true)

✅ Infrastructure initialized successfully
```

ใช้คำสั่ง **python manage.py init** ใน Terminal เพื่อเริ่มต้นระบบ.

ผลลัพธ์ที่ปรากฏในรูป:

- **Generating TLS Certificates**: ระบบทำการสร้างใบรับรองความปลอดภัย (HTTPS) ให้โดยอัตโนมัติ.
- **Network Isolation**: ระบบทำการตรวจสอบและสร้างเครือข่ายเสมือน (Virtual Network) สำหรับวง Private และ Public เพื่อแยกทราฟฟิกออกจากกัน.
- **Infrastructure initialized successfully**: ข้อความยืนยันว่าโครงสร้างพื้นฐานพร้อมสำหรับการ Deploy ขั้นต่อไปแล้ว.

```
🚀 Deploying Mockup Infrastructure Stack
   Bare Metal → L3 → L4 → L5/6 → L7
==================================================

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
[
    {
        "name": "private_net",
        "id": "8029c0a983f7e4f3f5e0b1689fc2ae98ee33e93e1f49de58df0db90bac79683a",
        "driver": "bridge",
        "network_interface": "podman2",
        "created": "2026-03-04T20:37:40.121546148+07:00",
        "subnets": [
            {
                "subnet": "10.89.1.0/24",
                "gateway": "10.89.1.1"
            }
        ],
        "ipv6_enabled": false,
        "internal": true,
        "dns_enabled": true,
        "ipam_options": {
            "driver": "host-local"
        }
    }
]
✅ Private network is isolated (internal: true)
🔧 Executing: podman-compose up -d --remove-orphans
mockup-gateway
```

Bash (Linux)

```shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```

🟡 VERIFY CONTAINERS

```none
podman ps
```

Should show:

Container	Port
stateless-api	3000

Container	Port
stateful-api	3001

```
Getting image source signatures
Copying blob sha256:955a8478f9aceb66cbf2f579fa3c24e1af278d1fa3ffd3043d6260e21d2f7416
Copying blob sha256:bca5d04786e112d958f100a66f8257b2aeefc14b64d81e405c3c44acff2fb000
Copying blob sha256:3e2c181db1b0985ce357c7aaf48ac615f30f392cd15d5c5ba34c4faa1f4f39a2
Copying blob sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153
Copying blob sha256:6b7b6c7061b76cdb8601e18722d12ae3232f0ddcfa1d2983754abcc6ce0a8a83
Copying blob sha256:399d0898a94e0084f81499a3e3c29824357118c7ce551648ea1dbab813884661
Copying blob sha256:6d397a54a185dd0b6638d1a3934b81daef7a140741e12697377d6279066f7ca7
Copying blob sha256:5e7756927bef33a266e1221356d5da8553139cb80bc5b1b3827010811d9ea268
Copying config sha256:b76de378d57272a1dd9091a05dd548a3639dfb792ebdbf95d06704d2950afdea
Writing manifest to image destination
229d5578e0092d629d0bdc4830a38b2c96f25e635a8aaa723dd3333bd95aa9ab
mockup-public-web
mockup-intranet-api
mockup-gateway
PS C:\network\automation\mockup-infra> podman ps
CONTAINER ID  IMAGE                                       COMMAND               CREATED        STATUS         PORTS                                       NAMES
11ae78d2f83e  localhost/mockup-infra_public_app:latest    python app.py         52 seconds ago Up 43 seconds  80/tcp                                      mockup-public-web
88a52a35a4be  localhost/mockup-infra_intranet_api:latest  python api.py         51 seconds ago Up 43 seconds  5000/tcp                                    mockup-intranet-api
229d5578e009  docker.io/library/nginx:alpine              nginx -g daemon o...  43 seconds ago Up 42 seconds  0.0.0.0:443->443/tcp, 0.0.0.0:8080->80/tcp  mockup-gateway
```

PUBLIC_NET_SUBNET=192.168.10.0/24: กำหนดวงเครือข่ายภายใน (Private LAN A).

HOST_IP=192.168.10.10: กำหนด IP Address ให้กับ ServerA.

REMOTE_NET_PUBLIC=10.10.12.0/30: ระบุวงเครือข่ายสาธารณะที่ใช้เชื่อมต่อไปยัง Site อื่น

มีการใช้คำสั่ง **python manage.py init** ซึ่งระบบดำเนินการดังนี้:

- **Generating TLS Certificates**: สร้างใบรับรองความปลอดภัยสำหรับการเชื่อมต่อ HTTPS.
- **Network Isolation**: สร้าง Docker Network เสมือนเพื่อจำลองการแยกวงเครือข่าย (Private vs Public).
- **Infrastructure initialized successfully**: ข้อความยืนยันว่าโครงสร้างพื้นฐานถูกเตรียมพร้อมสำหรับการ **Deploy** ตัว API ในขั้นตอนถัดไปเรียบร้อยแล้ว

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```shell
curl http://192.168.10.10:3000/info
```

→ Stateless

From ClientA:

```shell
curl -X POST http://192.168.10.10:3001/session
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check session persists across NAT

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
```

```
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```